3rd Year Project, Heating System Design
Diesel & Gas Burner
Done Using MAPLE 2025

==== Key Design Variables ====

```
> restart;
  with(ThermophysicalData):
>
  #All tunable Design variables shall live here. Nothing Bellow
  this section, should contain a hard coded number that might be
  changed. This section will be key for the later design
  optimization with MATLAB.

  #Citation: https://www.heatersim.com/knowledge/ReadArticle?
  API-560-Fired-Heater-Design-Approach-Article-2652023_162349_74826


> print("==== System Targets ====");
  Q__target := 68.856;        #kW  #total heat put to salt
  T__salt_in := 580.65;       #C  #Initial salt temperature
  T__salt_out := 620;     #C #Final salt tamperature
```

"==== System Targets ===="

$$Q_{target} := 68.856$$

$$T_{salt\_in} := 580.65$$

$$T_{salt\_out} := 620 \quad \textbf{(1.1)}$$

```
> print("==== Salt Properties ====");
  # Citations:
  # https://www.sciencedirect.
  com/science/article/pii/S0040603124000029#:~:text=
  long%2Dterm%20application.-,Abstract,
  heating%20and%20heating%2Dcooling%20cycles.
  # https://pubs.rsc.org/en/content/articlehtml/2016/fd/c5fd00236b
  # The best ratio of the mixture is a mass ratio with 30% Na2CO3,
  33% Li2CO3, 37% K2CO3.
  # All values for this mixture can be found in the above reports.

  T__salt_melt := 395.68;             # C
  T__salt_max := 650;                 # C
  Cp__salt := 2300;                   # J/kg/K   (Specific Heat)
```

```
rho__salt := 2050;                          # kg/m3    (Viscosity)
k__salt := 0.55;                            # W/m/K)
R__gas := 8.314;                            # J/mol/K
F_mu__salt := T_K -> 0.0852*exp((3.51e4)/(R__gas*T_K));  # mPa•s
```

"==== Salt Properties ===="

$$T_{salt\_melt} := 395.68$$

$$T_{salt\_max} := 650$$

$$Cp_{salt} := 2300$$

$$\rho_{salt} := 2050$$

$$k_{salt} := 0.55$$

$$R_{gas} := 8.314$$

$$F\_mu_{salt} := T\_K \mapsto 0.0852 \cdot e^{\frac{35100.}{R_{gas} \cdot T\_K}} \quad \textbf{(1.2)}$$

```
> print("==== Produced BioGas Properties ====");
xi__CH4 := 0.60;                            # mol fraction CH4 in biogas
xi__CO2 := 0.40;                            # mol fraction CO2 in biogas
m__gas_factory := 0.002125633;              # kg/s (factory biogas supply)
m__molar_CH4 := 16.04/1000;                 # kg/mol
m__molar_CO2 := 44.01/1000;                 # kg/mol
Q__mol_CH4 := 891;                          # kJ/mol (HHV)
```

"==== Produced BioGas Properties ===="

$$\xi_{CH4} := 0.60$$

$$\xi_{CO2} := 0.40$$

$$m_{gas\_factory} := 0.002125633$$

$$m_{molar\_CH4} := 0.01604000000$$

$$m_{molar\_CO2} := 0.04401000000$$

$$Q_{mol\_CH4} := 891 \quad \textbf{(1.3)}$$

```
> print("==== Imported BioGas Properties ====");
# Assumed same composition as factory biogas
# This is the unknown we solve for in script
m__gas_import := 0.00;                      # kg/s — initial guess,
updated later
```

"==== Imported BioGas Properties ===="

$$m_{gas\_import} := 0. \tag{1.4}$$

```
> print("==== Produced BioOil Properties ====");
  # Source:
  # https://www.sciencedirect.
  com/science/article/pii/S0960852403000592#aep-section-id11
  # https://www.researchgate.net/publication/324117596_Bio-
  Oil_Viscosity_of_Sisal_Residue_Process_and_Temperature_Influence
  # Empirical formula: CH_1.43 O_0.332 N_0.0013
  m__oil := 0.003486039;              # kg/s
  rho__oil := 1200;                   # kg/m3 (at 25°C)
  HHV__oil := 31030;                  # kJ/kg
  LHV__oil := 29360;                  # kJ/kg
  nu__oil := 60*10^(-6);              # m2/s (Kinematic viscosity)
  mu__oil := nu__oil * rho__oil ;     # Pa*s (Dynamic viscosity)
  sigma__oil := 30*10^(-3);           # N/m (Surface Tension)
  Q__comb_oil := m__oil * LHV__oil;   # kJ/mol

  w__C := 0.6393;                     # carbon
  w__H := 0.0761;                     # hydrogen
  w__O := 0.2836;                     # oxygen
  w__N := 0.0010;                     # nitrogen

  n__H := 1.43;                       # H/C ratio
  n__O := 0.332;                      # O/C ratio
  n__N := 0.0013;                     # N/C ratio
```

"==== Produced BioOil Properties ===="

$$m_{oil} := 0.003486039$$

$$\rho_{oil} := 1200$$

$$HHV_{oil} := 31030$$

$$LHV_{oil} := 29360$$

$$\nu_{oil} := \frac{3}{50000}$$

$$\mu_{oil} := \frac{9}{125}$$

$$\sigma_{oil} := \frac{3}{100}$$

$$Q_{comb\_oil} := 102.3501050$$

$$w_C := 0.6393$$

$$w_H := 0.0761$$

$$w_O := 0.2836$$

$$w_N := 0.0010$$

$$n_H := 1.43$$

$$n_O := 0.332$$

$$n_N := 0.0013 \quad \textbf{(1.5)}$$

```
> print("==== Gas Burner Design ====");
  # Sources:
  # https://ndl.ethernet.edu.
  et/bitstream/123456789/87824/4/Design%20of%20biogas%20burner.pdf
  # https://learning.oreilly.com/library/view/oxygen-enhanced-
  combustion-second/9781439862285/chapter-108.html

  T__prior_gas := 100 + 273.15;          # K — gas preheat
  temperature
  p__prior_gas := 30;                    # mBar — gas supply
  pressure
  C__d_gas := 0.9;                       # discharge coefficient
  (orifice)
  a__o_gas := 30;                        # degrees — orifice cone
  angle
  n__p_gas := 15;                         # number of flame ports
  d__p_gas := 0.005;                     # m — flame port diameter
```

"==== Gas Burner Design ===="

$$T_{prior\_gas} := 373.15$$

$$p_{prior\_gas} := 30$$

$$C_{d\_gas} := 0.9$$

$$a_{o\_gas} := 30$$

$$n_{p\_gas} := 15$$

$$d_{p\_gas} := 0.005 \quad \textbf{(1.6)}$$

```
> print("==== Diesel Burner Design ====");
  DP__oil := 10e5;                       # Pa    (Injection pressure
  drop)
  D__0 := 0.0015;                        # m     (Nozzle discharge
  diameter)
  D__p := 0.0006;                        # m     (Oil inlet to
  atomizer diameter)
  D__s := 0.003;                         # m     (Atomizer diameter)
  L__s := 0.002;                         # m     (Swirl Chamber
  Height)
```

```
L__0 := 0.001;                          # m     (Nozzle Height
L__p := 0.001;                          # m     (Pipe into swirl
length)
rho__a := 0.4;                          # kg/m3 (ambient air
density in firebox)(Double check for real temperature)
n__p_oil := 4;                          # (need to check)

#===Check1===

CheckRatio1 := proc(L__s, D__s)
    local ratio;
    ratio := evalf(L__s/D__s);
    if ratio > 0.5 then
        return "pass";
    else
        return "fail";
    end if;
end proc:

#===Check2===

CheckRatio2 := proc(L__p, D__p)
    local ratio;
    ratio := evalf(L__p/D__p);
    if ratio > 1.3 then
        return "pass";
    else
        return "fail";
    end if;
end proc:

Check__1 := CheckRatio1(L__s, D__s) = L__s/D__s;
Check__2 := CheckRatio2(L__p, D__p) = L__p/D__p;
Check__3 := L__0/D__0;
```

"==== Diesel Burner Design ===="

$$DP_{oil} := 1.0 \times 10^{6}$$

$$D_0 := 0.0015$$

$$D_p := 0.0006$$

$$D_s := 0.003$$

$$L_s := 0.002$$

$$L_0 := 0.001$$

$$L_p := 0.001$$

$$\rho_a := 0.4$$

$$n_{p\_oil} := 4$$

$$Check_1 := \text{"pass"} = 0.6666666667$$

$$Check_2 := \text{"pass"} = 1.666666667$$

$$Check_3 := 0.6666666667 \quad \textbf{(1.7)}$$

```
> print("==== Heater Design ====");
  # Source:
  # https://agushoe.wordpress.com/wp-content/uploads/2010/02/fired-
  heater.pdf

  #=Heater Geometry=

  #Radiation Tubes
  d__tube_OD_heat := 0.048;                          #m (Outside
  Diameter of Radiation Coil)
  d__tube_ID_heat := 0.038;                          #m (Internal
  Diameter of Radiation Coil)
  D__coil_heat := 1;                                 #m (Diameter of of
  Radiation Coil)
  p__coil_heat := 0.1075 ;                            #m (Pitch of
  Radiation Coil)
  D__shell_heat := 1.25;                               #m (Diameter of
  shell)

  #Convection Tubes
  d__tube_OD_conv := 0.035;                          #m (Outside
  Diameter of convection tube)
  d__tube_ID_conv := 0.027;                          #m (Inside
  Diameter of convection tube)
  p__conv_horizontal := 2.5 * d__tube_OD_conv;    #m (Spacing
  between centres of convection tubes)
  p__conv_vertical := 2.5 * d__tube_OD_conv;      #m (vertical
  spacing between centres of convection tubes)
  W__conv_bend := 0.005;                             #m (Width of space
  within the heater, taken by the bending of the convection coil)
  n__shield_rows := 2;                               #   (Number of
  layers which would experience the same amount of radiation)
  n__conv_parallel := 7;                             #   (Number of
  parallel streams within convection snake)

  #Heat coil Tube Material
```

```
k__tube_heat := 25;              #W/m/K (Thermal conductivity)
(Base calculations using a general stainless steel MUST ADAPT)

#Refactory Design

t__refr_heat := 0.15;            #m (Refactory lining thickness)
k__refr_heat := 0.35;            #W/m/K9 (Thermal conductivity)
(Just a base material/value, NEED to specify and detemrine the
best)
```

"==== Heater Design ===="

$$d_{tube\_OD\_heat} := 0.048$$

$$d_{tube\_ID\_heat} := 0.038$$

$$D_{coil\_heat} := 1$$

$$p_{coil\_heat} := 0.1075$$

$$D_{shell\_heat} := 1.25$$

$$d_{tube\_OD\_conv} := 0.035$$

$$d_{tube\_ID\_conv} := 0.027$$

$$p_{conv\_horizontal} := 0.0875$$

$$p_{conv\_vertical} := 0.0875$$

$$W_{conv\_bend} := 0.005$$

$$n_{shield\_rows} := 2$$

$$n_{conv\_parallel} := 7$$

$$k_{tube\_heat} := 25$$

$$t_{refr\_heat} := 0.15$$

$$k_{refr\_heat} := 0.35 \quad \textbf{(1.8)}$$

```
> print("==== Heat Coil Performance Targets ====");

#Performance Targets
z__excess := 0.15;          # Fraction of excess air, but will be
tuned to satisfy both air AFR requirements of both burners
eta__heater := 0.74;        # % Thermal efficeincy, is an
assumption, will be optimized/verified
f__radiant := 0.803;         # % fraction of heat absorbed in
```

```
radiant zone. Is an assumption, will be optimized/verified
q__rad := 10000;           # W/m2 Specific Radiant heat flux
```

"==== Heat Coil Performance Targets ===="

$$z_{excess} := 0.15$$

$$\eta_{heater} := 0.74$$

$$f_{radiant} := 0.803$$

$$q_{rad} := 10000 \quad \textbf{(1.9)}$$

==========

==========

==========

==========

==== Gas Burner ====

```
> #Work is based on these two sources
  #https://ndl.ethernet.edu.
  et/bitstream/123456789/87824/4/Design%20of%20biogas%20burner.pdf
  #https://learning.oreilly.com/library/view/oxygen-enhanced-
  combustion-second/9781439862285/chapter-108.html

> #As gas flow needs to eb fixed for efficient burner design, I
  will design requiring a specific amount of gas flow rate for
```

```
operations, which can be supplied with from the plant, and with a
top up tank if needed.
```

```
> #Fuel calculations

m__fuel_gas := m__gas_factory + m__gas_import;  #Sum of the two
gas flows
Q__mol_fuel := Q__mol_CH4 * xi__CH4; #kJ/mol        #Amount of
energy in mol of fuel
m__molar_fuel := (xi__CH4*m__molar_CH4) + (xi__CO2 *
m__molar_CO2); #kg/mol     #Molar mass of fuel
Q__comb_gas := (m__fuel_gas/m__molar_fuel)*Q__mol_fuel;  # kW
```

$$m_{fuel\_gas} := 0.002125633$$

$$Q_{mol\_fuel} := 534.60$$

$$m_{molar\_fuel} := 0.02722800000$$

$$Q_{comb\_gas} := 41.73510364 \tag{2.1}$$

```
> #Injector Design Calculations
```

```
> eq1 := V__fuel = 0.0467*C__d_gas * A__o*((p__prior_gas*10.197)/s)
^(1/2);
```

$$eq1 := V_{fuel} = 0.1342133139\, A_o \sqrt{30}\, \sqrt{\frac{1}{s}} \tag{2.2}$$

```
> #Variables
biogas := sprintf("CH4[%f]&CO2[%f]", xi__CH4, xi__CO2); #mixture
of biogas
#For C__d_gas, we must follow design optimization rules to make
this value valid.
rho__mix := CoolProp:-PropsSI("D","P",101325+(p__prior_gas*100),
"T", T__prior_gas,biogas); #kg/m3
m__fuel_kg_h := (m__fuel_gas * 3600);
V__fuel := m__fuel_kg_h / rho__mix; #m3/h
s := rho__mix/1.225;
```

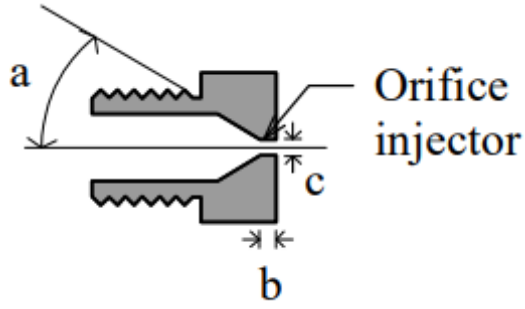
$$biogas := \text{"CH4[0.600000]\&CO2[0.400000]"}$$

$$\rho_{mix} := 0.916736826523549930$$

$$m_{fuel\_kg\_h} := 7.652278800$$

$$V_{fuel} := 8.347301623$$

$$s := 0.7483565930 \quad \textbf{(2.3)}$$




```
> #Injector Orifice Design (Based on paper's design rules)
  A__o := solve(eq1,A__o)/1000000;   #m2  #Area of Orifice
  r__o := (A__o/3.14)^(1/2); #m #Radius of Orifice
  b__o := 1.75*r__o; #m
  d__o := r__o*2;
```

$$A_o := 9.823000673 \times 10^{-6}$$

$$r_o := 0.001768712572$$

$$b_o := 0.003095247001$$

$$d_o := 0.003537425144 \quad \textbf{(2.4)}$$

```
> #Air to Fuel ratio

  # CH4 + 202 ==> CO2 + 2H20
  # As there is 21% oxygen in air, and the CH4 is in a mixture, the
  AFR will have ratios. There must also be 2:1 ratio of O2:CH4

  Per__CH4 := 1/xi__CH4;
  Per__O2 := 2/0.21;
  v__air := Per__O2/Per__CH4; #m3 air/ m3 gas
  V__air := v__air*V__fuel; #m3/h of air at atmosphere conditions
  r_ent := v__air*0.5; #Entrainment Ratio
```

$$Per_{CH4} := 1.666666667$$

$$Per_{O2} := 9.523809524$$

$$v_{air} := 5.714285713$$

$$V_{air} := 47.69886641$$

$$r\_ent := 2.857142856 \quad \textbf{(2.5)}$$

```
> #Throat Design Calculations
```

```
> #Variables

  v__o := V__fuel/(3600*A__o); #mm/s   #Velocity within the orifice
  v__t := v__o*(A__o/A__t); #mm/s
  p__o := 101325; #Pa
  p__t := p__o - rho__mix*(v__o^2/2)*(1-(d__o/d__t)^4); #Pa
```

$$v_o := 236.0475146$$

$$v_t := \frac{0.002318694895}{A_t}$$

$$p_o := 101325$$

$$p_t := 75785.43204 + \frac{3.999102910 \times 10^{-6}}{d_t^4} \tag{2.6}$$

```
> eq2 := r_ent = s^(1/2)*((d__t/d__o)-1);#Entrainment Ratio
  (Desired amount is roughly 50% of AFR
  #Prigg's Formula, #Prigg's formula holds if the total flame port
  area (Ap) is between 1.5 and 2.2 times the area of the throat.
```

$$eq2 := 2.857142856 = 244.5496446\, d_t - 0.8650760620 \tag{2.7}$$

```
> #Throtle Parameters

  d__t := solve(eq2, d__t); #mm #Diameter
  L__t := d__t * 10; #mm #Length
  A__t := 3.14*(d__t/2)^2;
```

$$d_t := 0.01522070876$$

$$L_t := 0.1522070876$$

$$A_t := 0.0001818609305 \tag{2.8}$$

```
> #Ensuring no backflow
  #Checking pressure drop, and ensuring enough pressure to push
  forward
```

```
> eq3 := Re__t = (4*rho__t*V__airfuel)/(3.14*mu__t*(d__t));
```

$$eq3 := \Re_t = \frac{83.69422016\, \rho_t\, V_{airfuel}}{\mu_t} \tag{2.9}$$

```
> V__airfuel := (V__fuel*(1 + r_ent))/3600; #m3/s  #converted into
  per s, and is now the volume flow rate of the two gasses
```

(2.10)

$$V_{airfuel} := 0.008943537450 \quad \textbf{(2.10)}$$

```
> #to find the pressure and the viscosity in the tube, I used a
  pseudo fluid, and the inputted thermodata sets, to do a lookup of
  the composition of CH4, CO2, N2 and O2.

  total_mols := 1 + r_ent;
```

$$total\_mols := 3.857142856 \quad \textbf{(2.11)}$$

```
> x__CH4_t := xi__CH4/total_mols;
  x__CO2_t := xi__CO2/total_mols;
  x__N2_t := (r_ent*0.79)/total_mols;
  x__O2_t := (r_ent*0.21)/total_mols;
```

$$x_{CH4\_t} := 0.1555555556$$

$$x_{CO2\_t} := 0.1037037037$$

$$x_{N2\_t} := 0.5851851852$$

$$x_{O2\_t} := 0.1555555555 \quad \textbf{(2.12)}$$

```
> pseudo_fluid := sprintf("CH4[%f]&CO2[%f]&Nitrogen[%f]&Oxygen[%f]
  ", x__CH4_t, x__CO2_t, x__N2_t, x__O2_t);
  m__air_g_s := m__fuel_gas*v__air*(1.184/rho__mix);
```

$$pseudo\_fluid := \text{"CH4[0.155556]\&CO2[0.103704]\&Nitrogen[0.585185]\&Oxygen[0.155556]"}$$

$$m_{air\_g\_s} := 0.01568762717 \quad \textbf{(2.13)}$$

```
> p__t := p__t;
  T__t := (m__fuel_gas*T__prior_gas + m__air_g_s*298.18)/
  (m__fuel_gas + m__air_g_s);
```

$$p_t := 75859.94350$$

$$T_t := 307.1260719 \quad \textbf{(2.14)}$$

```
> rho__t := CoolProp:-PropsSI("D", "P", p__t, "T", T__t,
  pseudo_fluid);
  mu__t  := CoolProp:-PropsSI("V", "P", p__t, "T", T__t,
  pseudo_fluid);
```

$$\rho_t := 0.845012941615553181$$

$$\mu_t := 0.000017032043214477 4462 \quad \textbf{(2.15)}$$

```
> Re__t := solve(eq3, Re__t);
  f__t := 0.316/(Re__t^(1/4));
  DP__t_gas := (f__t/2)*rho__t*((16*V__airfuel^2)/(3.14^2*(d__t)^5)
  )*(L__t); #Pa
```

$$\Re_t := 37136.53734$$

$$f_t := 0.02276337908$$

$$DP_{t\_gas} := 232.6001217 \quad \textbf{(2.16)}$$

```
> #Flame Port Calculation
  A__p_gas := n__p_gas *(3.14*d__p_gas^2/4);
  v__p_gas := V__airfuel/A__p_gas;
```

$$A_{p\_gas} := 0.0002943750000$$

$$v_{p\_gas} := 30.38144357 \quad \textbf{(2.17)}$$

==========

==========

==========

==========

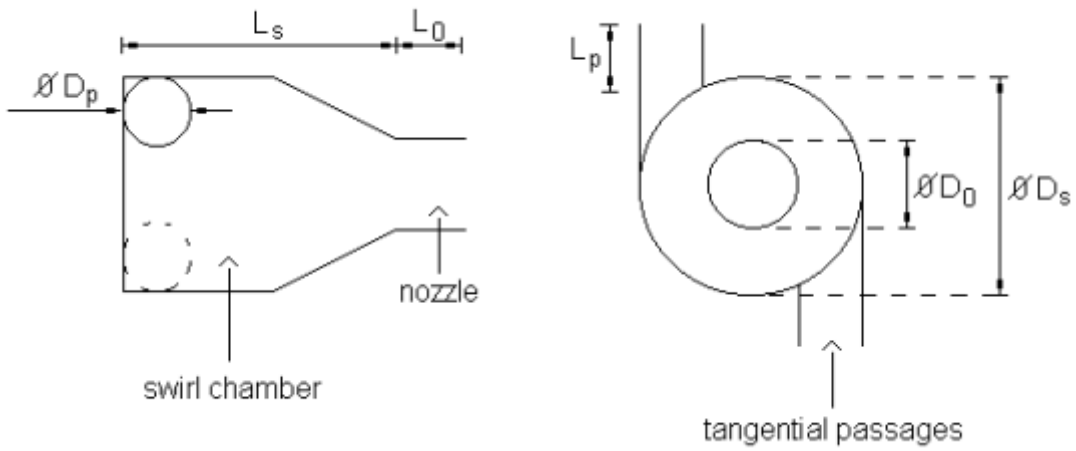
==== Oil Burner ====

```
> # Citation: Lacava, Bastos-Netto & Pimenta (2004)
  # "Design Procedure and Experimental Evaluation of
  #  Pressure-Swirl Atomizers"
  # 24th International Congress of the Aeronautical Sciences
```

```
> FN__oil := m__oil/(rho__oil * DP__oil)^(1/2); #m2
```

$$FN_{oil} := 1.006332778 \times 10^{-7} \quad \textbf{(3.1)}$$

>




```
> #Nozzle dimensions calculations
```

```
> A__0 := 3.14*(D__0/2)^2; #m2 (Cross section area of nozzle)
  Cd__oil := simplify(m__oil/(A__0 * (2*rho__oil*DP__oil)^(1/2)));
  #m2 (Discharge coefficient)
  A__p_oil := n__p_oil * 3.14*(D__p/2)^2;
```

$$A_0 := 1.766250000 \times 10^{-6}$$

$$Cd_{oil} := 0.04028788286$$

$$A_{p\_oil} := 1.130400000 \times 10^{-6} \qquad \textbf{(3.2)}$$

```
> #Calculating nozzle sizing ratios and their acceptance

  eq1__Carlisle_oil := Cd__oil = 0.0616*(D__s/D__0)*(A__p_oil/
  (D__s*D__0));
  eq2__RL_oil := Cd__oil = 0.35*(D__s/D__0)^0.5*(A__p_oil/(D__s*
  D__0))^0.5;
  eq3__Jones_oil := Cd__oil = 0.45*((D__0*rho__oil*U__0_oil)
  /mu__oil)^(-0.02)*(L__0/D__0)^(-0.03)*(L__s/D__s)^0.05*(A__p_oil/
  (D__s*D__0))^0.52*(D__s/D__0)^0.23;
```

$$eq1_{Carlisle\_oil} := 0.04028788286 = 0.03094784000$$

$$eq2_{RL\_oil} := 0.04028788286 = 0.2480806320$$

$$eq3_{Jones\_oil} := 0.04028788286 = \frac{0.2393200019}{U_{0\_oil}^{0.02}} \qquad \textbf{(3.3)}$$

```
> #Acceptance based on spray semi angle

  K__oil := A__p_oil/(D__s*D__0);
  eqX := D__0 = 2*((FN__oil)/(3.14*(1-X__oil)*(2)^(1/2)))^(1/2);
  X__oil := solve(eqX,X__oil);
  eq3__oil := sin(theta__oil) = ((3.14/2)*Cd__oil)/(K__oil*(1+
```

```
(X__oil)^(1/2)));
theta__oil := simplify(solve(eq3__oil,theta__oil));
```

$$K_{oil} := 0.2512000000$$

$$eqX := 0.0015 = 0.0002531750959\sqrt{\frac{\sqrt{2}}{1 - X_{oil}}}$$

$$X_{oil} := 0.9597121171$$

$$eq3_{oil} := \sin\left(\theta_{oil}\right) = 0.1271938970$$

$$\theta_{oil} := 0.1275393814 \quad \textbf{(3.4)}$$

```
> #Liquid Sheet Thickness at Nozzle tip

h__0_oil := simplify((0.00805*FN__oil*(rho__oil)^(1/2))/(D__0*cos
(theta__oil)));
```

$$h_{0\_oil} := 0.00001886160569 \quad \textbf{(3.5)}$$

```
> #Calculating the ligament diameter

U__0_oil := simplify((2*DP__oil/rho__oil)^(1/2));
D__L_oil := simplify( 0.9615*cos(theta__oil)*((h__0_oil^4*
sigma__oil^2)/(U__0_oil^4*rho__a*rho__oil))^(1/6)*(1+2.6*mu__oil*
cos(theta__oil)*((h__0_oil^2*rho__a^4*U__0_oil^7)/(72*rho__oil^2*
sigma__oil^5)^(1/3)))^0.2);
```

$$U_{0\_oil} := 40.82482904$$

$$D_{L\_oil} := 6.601909413 \times 10^{-6} \quad \textbf{(3.6)}$$

```
> # Mean Diameter Size
#If the atomizer correctly forms droplets, then droplet diameter
is:

SMD_oil := 1.89*D__L_oil;
```

$$SMD\_oil := 0.00001247760879 \quad \textbf{(3.7)}$$

==========

==========

==========

==========

==== Heating Coils ====

```
> #System Requirements
```

```
> Q__absorbed := Q__target*1000; #W (Amount of energy to the salt)
  Q__released := Q__target*1000/eta__heater; #W (Amount of energy
  released by the Oil AND Gas burners)
  Q__radiant := f__radiant * Q__absorbed; #W (Amount of energy
  absorbed through radiation)
  Q__convection := Q__absorbed - Q__radiant; #W (Amount of energy
  absorbed through convection)
  m__salt := (Q__absorbed)/(Cp__salt * (T__salt_out - T__salt_in));
  #kg/s
```

$$Q_{absorbed} := 68856.000$$
$$Q_{released} := 93048.64865$$
$$Q_{radiant} := 55291.36800$$
$$Q_{convection} := 13564.63200$$
$$m_{salt} := 0.7607977459 \quad \textbf{(4.1)}$$

```
> #As raditation primarily comes from flue gas, the mass flow rate
  of flue gas is required
```

```
> #Flue gas from gas burners
  #
  O2__per_kg_gas := (xi__CH4 / m__molar_fuel) * 2 * 0.032;
  air__per_kg_gas := O2__per_kg_gas / 0.233; # kg air per kg gas
```

$$O2_{per\_kg\_gas} := 1.410312913$$
$$air_{per\_kg\_gas} := 6.052845121 \quad \textbf{(4.2)}$$

```
> #Flue gas from oil burners

  O2__per_kg_oil := (w__C/12)*32 + (w__H/4)*32 - (w__O/32)*32;
  air__per_kg_oil := O2__per_kg_oil / 0.233;  # kg air per kg oil
```

$$O2_{per\_kg\_oil} := 2.030000000$$
$$air_{per\_kg\_oil} := 8.712446353 \quad \textbf{(4.3)}$$

```
> m__total_air := (1+z__excess)*((air__per_kg_gas * m__fuel_gas) +
  (air__per_kg_oil * m__oil)); #kg/s
```

```
m__flue := m__total_air + m__fuel_gas + m__oil; #kg/s
```

$$m_{total\_air} := 0.04972376336$$

$$m_{flue} := 0.05533543536 \quad \textbf{(4.4)}$$

```
> # Calculating how much energy is absorbed from
  radiation/convection

  Q__radiant := f__radiant * Q__absorbed; #W ,Initial guess of
  f__radiant, f__radiant needs to be converged.
```

$$Q_{radiant} := 55291.36800 \quad \textbf{(4.5)}$$

Radiated Segment

```
> # ==== Calculating coil Geometry ====

  print("====Turn calculations====");
  A__radiant := Q__radiant/q__rad; #m2 (Required surface area)
  L__turn_heat := sqrt((Pi*D__coil_heat)^2 + p__coil_heat^2); #m
  A__per_turn := Pi * d__tube_OD_heat * L__turn_heat; #m2
  N__turns_heat := A__radiant / A__per_turn;
  N__turns_int_heat := ceil(N__turns_heat);
  H__coil := N__turns_int_heat * p__coil_heat; #m (total hight of
  heater column)
```

"====Turn calculations===="

$$A_{radiant} := 5.529136800$$

$$L_{turn\_heat} := 3.143431350$$

$$A_{per\_turn} := 0.4740182802$$

$$N_{turns\_heat} := 11.66439572$$

$$N_{turns\_int\_heat} := 12$$

$$H_{coil} := 1.2900 \quad \textbf{(4.1.1)}$$

```
> # Key final geometry values

  A__rad_actual := N__turns_int_heat * A__per_turn; #m2 (Calculates
  area with rounding up number of turns)
  q__rad := q__rad;
```

```
q__rad_actual := Q__radiant / A__rad_actual; # W/m2
L__tube_rad := N__turns_int_heat * L__turn_heat; # m (tube length
within radiated zone)
```

$$A_{rad\_actual} := 5.688219362$$

$$q_{rad} := 10000$$

$$q_{rad\_actual} := 9720.329770$$

$$L_{tube\_rad} := 37.72117620 \qquad \textbf{(4.1.2)}$$

```
> #Heating calculations of coil

A__cp := Pi * D__coil_heat * H__coil; #m2 (Area of cold plane)
x__spacing := p__coil_heat / d__tube_OD_heat;
alpha__tube := 1 - (0.0277 + 0.0927*(x__spacing - 1))*(x__spacing
- 1); #the ratio of tube vs refactory
```

$$A_{cp} := 4.052654524$$

$$x_{spacing} := 2.239583333$$

$$\alpha_{tube} := 0.8232237956 \qquad \textbf{(4.1.3)}$$

```
> # ==== Calculating Refactory Dimensions ====

> A__shell := Pi*D__shell_heat*H__coil + 2*(Pi/4)*D__shell_heat^2;
#m2
A__refr := A__shell - A__cp;    # m2 refractory area
```

$$A_{shell} := 7.520187416$$

$$A_{refr} := 3.467532892 \qquad \textbf{(4.1.4)}$$

```
> alpha__AR := evalf(alpha__tube * A__cp);    # m2 (Effective
absorbing area)
ratio__Aw_alphaAR := evalf(A__refr / alpha__AR); # Ratio of
refractory to effective absorbing area
```

$$\alpha_{AR} := 3.336241640$$

$$ratio_{Aw\_alphaAR} := 1.039353040 \qquad \textbf{(4.1.5)}$$

```
> # ==== Verifying system heat transfer ====

>
```

5. $d \times d$ — 2/3 diameter
6. $d \times 2d$ to $d \times \infty d$ — 1 × diameter

```
> D__firebox := evalf(D__coil_heat - d__tube_OD_heat);  #m (coil
inner diameter)
L_D__ratio := evalf(H__coil / D__firebox); #Ratio of hight of
coil vs diameter of firebox
```

$$D_{firebox} := 0.952$$

$$L\_D_{ratio} := 1.355042017 \quad \textbf{(4.1.6)}$$

```
> # mean beam length

  if L_D__ratio <= 1.0 then
      L__beam := evalf((2/3) * D__firebox); #m
  else
      L__beam := evalf(1.0 * D__firebox); #m
  end if;
```

$$L_{beam} := 0.9520 \quad \textbf{(4.1.7)}$$

```
> P__CO2_H2O := 0.288 - 0.229*z__excess + 0.090*z__excess^2;    #
  atm
```

$$P_{CO2\_H2O} := 0.2556750 \quad \textbf{(4.1.8)}$$

```
> # Must convert beam length to feet for Walas correlations
  L__beam_ft := evalf(L__beam * 3.28084);  # convert m to ft
  PL := P__CO2_H2O * L__beam_ft;    # atm*ft
```

$$L_{beam\_ft} := 3.123359680$$

$$PL := 0.7985649862 \quad \textbf{(4.1.9)}$$

```
> # Mean tube wall temperature

  # (AI generated equations) Converting salt temperatures to
  Fahrenheit for correlation, Annoying Americans >:(
  T__salt_in_F := evalf(T__salt_in * 9/5 + 32);
  T__salt_out_F := evalf(T__salt_out * 9/5 + 32);
```

$$T_{salt\_in\_F} := 1077.170000$$

$$T_{salt\_out\_F} := 1148. \quad \textbf{(4.1.10)}$$

```
> T__salt_rad_in_F := T__salt_in_F + (1 - f__radiant)*
  (T__salt_out_F - T__salt_in_F);
```

$$T_{salt\_rad\_in\_F} := 1091.123510 \quad \textbf{(4.1.11)}$$

```
> # Walas approximation for mean tube wall temperature
  T__tube_F := 100 + 0.5*(T__salt_rad_in_F + T__salt_out_F);  # F
  T__tube_R := T__tube_F + 460;                               #
  Rankine
  T__tube_C := evalf((T__tube_F - 32) * 5/9);                 # °C
  T__tube_K := evalf(T__tube_C + 273.15);                     # K
```

$$T_{tube\_F} := 1219.561755$$

$$T_{tube\_R} := 1679.561755$$

$$T_{tube\_C} := 659.7565305$$

$$T_{tube\_K} := 932.9065305 \quad \textbf{(4.1.12)}$$

```
> # ==== Iterative Calculation ====
```

```
#As the equation outlined in step 22 of the plan, requires the
calculation to be done iteratively, the approach that will be
done here, and should translate well into MATLAB is to sovle each
side of the combined equation, and to then adjust values until
they converge. For this section, Claude, opus4.6 was used to help
with setting up the equations for the iterations.
```

```
> # Convert to Imperial for Walas correlations
  Q__n_Btu := Q__released * 3.41214;   # This is Q_n for Walas
  equations
  Q__radiant_Btu := Q__radiant * 3.41214;       # W -> Btu/hr
  alphaAR__sqft := alpha__AR * 10.7639;        # m2 -> sqft
```

$$Q_{n\_Btu} := 317495.0160$$

$$Q_{radiant\_Btu} := 188661.8884$$

$$alphaAR_{sqft} := 35.91097139 \qquad \textbf{(4.1.13)}$$

```
> # Walas Eq. (4): Flue gas enthalpy ratio as function of T (°F)
  # Q_g/Q_n = [a + b*(T/1000-0.1)] * (T/1000 - 0.1)
  # where z = fraction excess air
  F_Qg_ratio := proc(T_F, z_ex)
      local a4, b4, t;
      t := T_F/1000 - 0.1;
      a4 := 0.22048 - 0.35027*z_ex + 0.92344*z_ex^2;
      b4 := 0.016086 + 0.29393*z_ex - 0.48139*z_ex^2;
      return (a4 + b4*t)*t;
  end proc:
  # Walas Eq. (8): Gas emissivity
  F_emissivity := proc(T_g_F, PL_val)
      local z8, a8, b8, c8;
      z8 := (T_g_F + 460)/1000;
      a8 := 0.47916 - 0.19847*z8 + 0.022569*z8^2;
      b8 := 0.047029 + 0.0699*z8 - 0.01528*z8^2;
      c8 := 0.000803 - 0.00726*z8 + 0.001597*z8^2;
      return a8 + b8*PL_val + c8*PL_val^2;
  end proc:
  # Walas Eq. (9): Exchange factor F
  F_exchange := proc(phi_val, z9)
      local a9, b9, c9;
      a9 := 0.00064 + 0.0591*z9 + 0.00101*z9^2;
      b9 := 1.0256 + 0.4908*z9 - 0.058*z9^2;
      c9 := -0.144 - 0.552*z9 + 0.040*z9^2;
      return a9 + b9*phi_val + c9*phi_val^2;
  end proc:
>
  #Start of itterative equations
```

```
T__g_guess := 1600;  # °F initial guess
f__loss := 0.025;
```

$$T_{g\_guess} := 1600$$

$$f_{loss} := 0.025 \qquad \textbf{(4.1.14)}$$

```
> # Newton-Raphson iteration
T__g := T__g_guess;
for iter from 1 to 30 do

    phi__g := F_emissivity(T__g, PL);
    z9__val := A__refr*10.7639 / alphaAR__sqft;
    F__exc := F_exchange(phi__g, z9__val);    # Exchange factor

    # === LHS: radiant heat transfer (Eq. 1) ===
    LHS := 1730*((T__g + 460)/1000)^4 - 1730*(T__tube_R/1000)^4 +
7*(T__g - T__tube_F);
    Q__R_calc := alphaAR__sqft * F__exc * LHS;  # Btu/hr (Actual
Q_R from LHS)

    # === RHS: from heat balance ===
    Qg_ratio := F_Qg_ratio(T__g, z__excess);
    Q__R_balance := Q__n_Btu * (1 - f__loss - Qg_ratio);  #
Btu/hr

    # Residual
    residual := Q__R_calc - Q__R_balance;

    # Numerical derivative (for Newton step)
    dT := 1.0;
    phi__g2 := F_emissivity(T__g + dT, PL);
    F__exc2 := F_exchange(phi__g2, z9__val);
    LHS2 := 1730*((T__g + dT + 460)/1000)^4 - 1730*
(T__tube_R/1000)^4 + 7*(T__g + dT - T__tube_F);
    Q__R_calc2 := alphaAR__sqft * F__exc2 * LHS2;
    Qg_ratio2 := F_Qg_ratio(T__g + dT, z__excess);
    Q__R_balance2 := Q__n_Btu * (1 - f__loss - Qg_ratio2);
    residual2 := Q__R_calc2 - Q__R_balance2;

    dres_dT := (residual2 - residual)/dT;

    if abs(dres_dT) < 1e-10 then break; end if;

    T__g_new := evalf(T__g - residual/dres_dT);
```

```
    if abs(T__g_new - T__g) < 0.01 then
        T__g := T__g_new;
        break;
    end if;
    T__g := T__g_new;
end do;
```

$$T_g := 1600$$

$$\phi_g := 0.2621457248$$

$$z9_{val} := 1.039353040$$

$$F_{exc} := 0.4029603548$$

$$LHS := 20050.44644$$

$$Q_{R\_calc} := 290143.9506$$

$$Qg\_ratio := 0.3940998562$$

$$Q_{R\_balance} := 184432.9005$$

$$residual := 105711.0501$$

$$dT := 1.0$$

$$\phi_{g2} := 0.2620453637$$

$$F_{exc2} := 0.4028480021$$

$$LHS2 := 20117.98386$$

$$Q_{R\_calc2} := 291040.0944$$

$$Qg\_ratio2 := 0.3944366551$$

$$Q_{R\_balance2} := 184325.9685$$

$$residual2 := 106714.1259$$

$$dres\_dT := 1003.075800$$

$$T_{g\_new} := 1494.613099$$

$$T_g := 1494.613099$$

$$\phi_g := 0.2728501150$$

$$z9_{val} := 1.039353040$$

$$F_{exc} := 0.4148657585$$

$$LHS := 13410.27878$$

$$Q_{R\_calc} := 199789.4495$$

$$Qg\_ratio := 0.3591588989$$

$$Q_{R\_balance} := 195526.4802$$

$$residual := 4262.9693$$

$$dT := 1.0$$

$$\phi_{g2} := 0.2727473543$$

$$F_{exc2} := 0.4147522031$$

$$LHS2 := 13468.99440$$

$$Q_{R\_calc2} := 200609.2836$$

$$Qg\_ratio2 := 0.3594852973$$

$$Q_{R\_balance2} := 195422.8504$$

$$residual2 := 5186.4332$$

$$dres\_dT := 923.4639000$$

$$T_{g\_new} := 1489.996818$$

$$T_g := 1489.996818$$

$$\phi_g := 0.2733247829$$

$$z9_{val} := 1.039353040$$

$$F_{exc} := 0.4153901042$$

$$LHS := 13140.25786$$

$$Q_{R\_calc} := 196014.0432$$

$$Qg\_ratio := 0.3576534311$$

$$Q_{R\_balance} := 196004.4588$$

$$residual := 9.5844$$

$$dT := 1.0$$

$$\phi_{g2} := 0.2732219170$$

$$F_{exc2} := 0.4152764984$$

$$LHS2 := 13198.60802$$

$$Q_{R\_calc2} := 196830.6098$$

$$Qg\_ratio2 := 0.3579793741$$

$$Q_{R\_balance2} := 195900.9735$$

$$residual2 := 929.6363$$

$$dres\_dT := 920.0519000$$

$$T_{g\_new} := 1489.986401$$

$$T_g := 1489.986401$$

$$\phi_g := 0.2733258546$$

$$z9_{val} := 1.039353040$$

$$F_{exc} := 0.4153912878$$

$$LHS := 13139.65042$$

$$Q_{R\_calc} := 196005.5404$$

$$Qg\_ratio := 0.3576500363$$

$$Q_{R\_balance} := 196005.5366$$

$$residual := 0.0038$$

$$dT := 1.0$$

$$\phi_{g2} := 0.2732229885$$

$$F_{exc2} := 0.4152776818$$

$$LHS2 := 13197.99978$$

$$Q_{R\_calc2} := 196822.1000$$

$$Qg\_ratio2 := 0.3579759781$$

$$Q_{R\_balance2} := 195902.0517$$

$$residual2 := 920.0483$$

$$dres\_dT := 920.0445000$$

$$T_{g\_new} := 1489.986397 \qquad \textbf{(4.1.15)}$$

```
> print("==== Converged Values ====");

  # Final values at converged T_g
  phi__g_final := F_emissivity(T__g, PL);
  F__exc_final := F_exchange(phi__g_final, z9__val);
  Q__R_final_Btu := alphaAR__sqft * F__exc_final * (1730*(
  (T__g+460)/1000)^4 - 1730*(T__tube_R/1000)^4 + 7*(T__g -
  T__tube_F));
  Q__R_final := Q__R_final_Btu / 3.41214;   # convert to W
  T__g_C := (T__g - 32)*5/9; #Celcius
```

"==== Converged Values ===="

$$\phi_{g\_final} := 0.2733258551$$

$$F_{exc\_final} := 0.4153912882$$

$$Q_{R\_final\_Btu} := 196005.5374$$

$$Q_{R\_final} := 57443.58009$$

$$T_{g\_C} := 809.9924428 \qquad \textbf{(4.1.16)}$$

```
> # ====V erification of q__rad ====

  q__rad_check := Q__R_final / A__rad_actual;   # W/m2
  q__rad_check_Btu := q__rad_check * 0.316998;  # W/m2 ->
```

```
  Btu/hr/ft2

  printf(("Actual radiant flux: %.0f W/m2 (%.0f Btu/hr/ft2)\n",
  q__rad_check), q__rad_check_Btu);
  printf("Target was: %.0f W/m2\n", q__rad);

  if abs(q__rad_check - q__rad)/q__rad > 0.10 then
      printf("  STATUS: >10%% deviation -- adjust coil geometry and
  re-run\n");
  else
      printf("  STATUS: PASS (within 10%% of target)\n");
  end if;
```

$$q_{rad\_check} := 10098.69283$$

$$q_{rad\_check\_Btu} := 3201.265430$$

```
Actual radiant flux: 10099 W/m2 (3201 Btu/hr/ft2)
Target was: 10000 W/m2
  STATUS: PASS (within 10% of target)
```

```
> # ==== Exploring salt side thermodynamics with designed geometry
  ====
```

```
> T__g_K := T__g_C + 273.15;
  T__salt_avg_K := (T__salt_in + T__salt_out)/2 + 273.15;
  mu__salt_Pas := F_mu__salt(T__salt_avg_K) / 1000;
  v__salt := m__salt / (rho__salt * Pi * (d__tube_ID_heat/2)^2);  #
  m/s
  Re__salt := rho__salt * v__salt * d__tube_ID_heat / mu__salt_Pas;
  Pr__salt := Cp__salt * mu__salt_Pas / k__salt;
  De__salt := Re__salt * sqrt(d__tube_ID_heat / D__coil_heat);
```

$$T_{g\_K} := 1083.142443$$

$$T_{salt\_avg\_K} := 873.4750000$$

$$\mu_{salt\_Pas} := 0.01070357753$$

$$v_{salt} := 0.3272338947$$

$$\Re_{salt} := 2381.588803$$

$$Pr_{salt} := 44.76041513$$

$$De_{salt} := 464.2571255 \qquad \textbf{(4.1.17)}$$

```
> # calculating Nusselt number

  if Re__salt > 10000 then
```

```
    Nu__salt := 0.023 * Re__salt^0.85 * Pr__salt^0.4 * (1 + 3.6*
(1 - d__tube_ID_heat/D__coil_heat)^0.8);
elif Re__salt > 2100 then
    Nu__salt := 0.023 * Re__salt^0.85 * Pr__salt^0.4 *
(d__tube_ID_heat/D__coil_heat)^0.1;# Uses Seban & McLaughlin
else
    Nu__salt := 0.913 * De__salt^(1/3) * Pr__salt^(1/3); # Dean
correlation
end if;
```

$$N_{salt} := 56.28602780 \qquad \textbf{(4.1.18)}$$

```
> h__i := Nu__salt * k__salt / d__tube_ID_heat; # W/m2/K (Inside
film coefficient)(energy transfered per m2 per kelvin difference)
```

$$h_i := 814.6661918 \qquad \textbf{(4.1.19)}$$

```
> # ==== Exploring heater side thermodynamics with designed
geometry ====


T__tube_K := (T__tube_F - 32)*5/9 + 273.15;
h__o := q__rad_check / (T__g_K - T__tube_K);   # W/m2/K (outisde
film coefficient) (The flux over the temperature difference)
```

$$T_{tube\_K} := 932.9065305$$

$$h_o := 67.21890034 \qquad \textbf{(4.1.20)}$$

```
> # ==== Exploring the thermodynamics within the designed geometry
===

> R__o := 1/h__o; #Thermal resistance on the outside
R__wall := (d__tube_OD_heat/2)*ln
(d__tube_OD_heat/d__tube_ID_heat)/k__tube_heat; #Thermal
resistance within the pipe
R__i := (d__tube_OD_heat/d__tube_ID_heat) * (1/h__i); #Thermal
resistance on the inside
U__overall := 1/(R__o + R__wall + R__i);   # W/m2/K (Overall heat
transfer coefficient)
```

$$R_o := 0.01487676821$$

$$R_{wall} := 0.0002242702574$$

$$R_i := 0.001550522052$$

$$U_{overall} := 60.05443146 \qquad \textbf{(4.1.21)}$$

```
> # ==== Temperature differences along the coil length ====

DT__hot := T__g_K - (T__salt_out + 273.15); #C or K
```

```
DT__cold := T__g_K - (T__salt_rad_in_F - 32)*5/9 - 273.15; #C or
K
T__salt_rad_in_C := (T__salt_rad_in_F - 32)*5/9;
```

$$DT_{hot} := 189.992443$$

$$DT_{cold} := 221.5904930$$

$$T_{salt\_rad\_in\_C} := 588.4019500 \quad \textbf{(4.1.22)}$$

```
> if abs(DT__hot - DT__cold) < 1 then
    LMTD__rad := (DT__hot + DT__cold)/2;
else
    LMTD__rad := (DT__hot - DT__cold)/ln(DT__hot/DT__cold);
end if;
```

$$LMTD_{rad} := 205.3865229 \quad \textbf{(4.1.23)}$$

```
> #checking values

A__required_UA := Q__radiant / (U__overall * LMTD__rad);
T__skin_max := T__salt_out + q__rad_check/h__i;  # C
T__skin_outside := T__skin_max + q__rad_check * R__wall;  # C
```

$$A_{required\_UA} := 4.482706789$$

$$T_{skin\_max} := 632.3961114$$

$$T_{skin\_outside} := 634.6609478 \quad \textbf{(4.1.24)}$$

```
> if T__skin_max > T__salt_max then #claude helped with notation
    printf(("  SKIN TEMP CHECK: FAIL (%.1f > %.1f C max)\n",
T__skin_max), T__salt_max);
    printf("  --> Reduce flux or increase h_i (more turns,
smaller tube)\n");
else
    printf(("  SKIN TEMP CHECK: PASS (%.1f < %.1f C max)\n",
T__skin_max), T__salt_max);
end if;
SKIN TEMP CHECK: PASS (632.4 < 650.0 C max)
```

Convection Segment

```
> Q__convection := Q__convection; #W
```

```
Q__convection_btu := Q__convection*3.41214; #Btu
```

$$Q_{convection} := 13564.63200$$

$$Q_{convection\_btu} := 46284.42343 \quad \textbf{(4.2.1)}$$

```
> #Unit Conversion

  d__tube_OD_in := d__tube_OD_conv / 0.0254;

  #Convection Duct design

  W__conv := evalf(((D__shell_heat^2)/2)^(1/2)); #m (Width of
  square duct that fits within shell circular diameter)
  n__tubes_per_row_conv := ceil(W__conv / p__conv_horizontal); #
  Number of tubes
  L__tube_conv:= W__conv - 2*W__conv_bend;
```

$$d_{tube\_OD\_in} := 1.377952756$$

$$W_{conv} := 0.8838834762$$

$$n_{tubes\_per\_row\_conv} := 11$$

$$L_{tube\_conv} := 0.8738834762 \quad \textbf{(4.2.2)}$$

```
> A__conv_free := simplify((W__conv * L__tube_conv) -
  (n__tubes_per_row_conv * d__tube_OD_conv * L__tube_conv)); #Not
  Including bends
  A__conv_free_sqft := A__conv_free * 10.7639; #ft^2
  G__convection := (m__flue * 2.20462) / A__conv_free_sqft; #Should
  fall between 0.3 - 0.4
```

$$A_{conv\_free} := 0.4359660264$$

$$A_{conv\_free\_sqft} := 4.692694712$$

$$G_{convection} := 0.02599649347 \quad \textbf{(4.2.3)}$$

```
> Qs__ratio := 1 - eta__heater + f__loss;

  # Iterate to find T_stack (Claude helped with loop function
  formation)
  T__stack := 500; # °F initial guess
  for iter2 from 1 to 30 do
     Qs__calc := F_Qg_ratio(T__stack, z__excess);
     residual2 := Qs__calc - Qs__ratio;
     Qs__calc2 := F_Qg_ratio(T__stack + 1, z__excess);
     dres2 := Qs__calc2 - Qs__calc;
     if abs(dres2) < 1e-10 then break; end if;
     T__stack := T__stack - residual2/dres2;
     if abs(residual2) < 0.0001 then break; end if;
  end do;
```

$$Qs_{ratio} := 0.285$$

$$T_{stack} := 500$$

$$Qs_{calc} := 0.08338183600$$

$$residual2 := -0.2016181640$$

$$Qs_{calc2} := 0.08361007761$$

$$dres2 := 0.00022824161$$

$$T_{stack} := 1383.354109$$

$$Qs_{calc} := 0.3234604375$$

$$residual2 := 0.0384604375$$

$$Qs_{calc2} := 0.3237758560$$

$$dres2 := 0.0003154185$$

$$T_{stack} := 1261.419481$$

$$Qs_{calc} := 0.2857396729$$

$$residual2 := 0.0007396729$$

$$Qs_{calc2} := 0.2860430578$$

$$dres2 := 0.0003033849$$

$$T_{stack} := 1258.981413$$

$$Qs_{calc} := 0.2850004133$$

$$residual2 := 4.133 \times 10^{-7}$$

$$Qs_{calc2} := 0.2853035576$$

$$dres2 := 0.0003031443$$

$$T_{stack} := 1258.980050 \qquad \textbf{(4.2.4)}$$

```
> #LMTD calculation In Farenheight
  DT__conv_hot := T__g - T__salt_rad_in_F;
  DT__conv_cold := T__stack - T__salt_in_F;

  if DT__conv_cold <= 0 then
      printf("  WARNING: Stack temp below salt inlet -- check
  efficiency target\n");
      LMTD__conv_F := DT__conv_hot;  # fallback
  elif abs(DT__conv_hot - DT__conv_cold) < 1 then
      LMTD__conv_F := (DT__conv_hot + DT__conv_cold)/2;
  else
      LMTD__conv_F :=(DT__conv_hot - DT__conv_cold)/ln
  (DT__conv_hot/DT__conv_cold);
  end if;
```

$$DT_{conv\_hot} := 398.862887$$

$$DT_{conv\_cold} := 181.810050$$

$$LMTD_{conv\_F} := 276.2698213 \quad \textbf{(4.2.5)}$$

```
> #LMTD calculation In Kelvin

  LMTD__conv_K := LMTD__conv_F * 5/9;
```

$$LMTD_{conv\_K} := 153.4832341 \quad \textbf{(4.2.6)}$$

```
> T__f_conv_F := 0.5* (T__salt_in_F + T__salt_rad_in_F) + 0.5*
  LMTD__conv_F
```

$$T_{f\_conv\_F} := 1222.281666 \quad \textbf{(4.2.7)}$$

```
> z__Uc := T__f_conv_F / 1000;

  a__Uc := 2.461 - 0.759*z__Uc + 1.625*z__Uc^2;
  b__Uc := 0.7655 + 21.373*z__Uc - 9.6625*z__Uc^2;
  c__Uc := 9.7938 - 30.809*z__Uc + 14.333*z__Uc^2;

  U__convection_Btu := simplify((a__Uc + b__Uc*G__convection +
  c__Uc*G__convection^2)*(4.5/d__tube_OD_conv)^0.25); #Btu/hr/ft2/F
  U__convection := U__convection_Btu * 5.67826;   #  W/m2/K
```

$$z_{Uc} := 1.222281666$$

$$a_{Uc} := 3.960993481$$

$$b_{Uc} := 12.45381705$$

$$c_{Uc} := -6.45036842$$

$$U_{convection\_Btu} := 14.41349946$$

$$U_{convection} := 81.84359744 \quad \textbf{(4.2.8)}$$

```
> #Dimension calculations of required convection area

  A__convection := Q__convection /(U__convection*LMTD__conv_K);
  #m^2
  A__conv_per_row := simplify(n__tubes_per_row_conv * 3.14 *
  d__tube_OD_conv * L__tube_conv);
  n__rows_conv := ceil(A__convection/A__conv_per_row);
  L__tube_conv_total := simplify(n__rows_conv *
  n__tubes_per_row_conv * (L__tube_conv+W__conv_bend));
```

$$A_{convection} := 1.079847342$$

$$A_{conv\_per\_row} := 1.056437734$$

$$n_{rows\_conv} := 2$$

$$L_{tube\_conv\_total} := 19.33543648 \quad \textbf{(4.2.9)}$$

```
> Q__convection_check := m__salt *Cp__salt * (T__salt_rad_in_C -
  T__salt_in);
  Q__convection:= Q__convection;
```

$$Q_{convection\_check} := 13564.63200$$

$$Q_{convection} := 13564.63200 \quad \textbf{(4.2.10)}$$

```
> #=== Thermo Dynamic reset for different temperatures ===

  T__salt_avg_conv := (T__salt_in + T__salt_rad_in_C)/2;
  T__salt_avg_conv_K := T__salt_avg_conv + 273.15;

  mu__salt_conv_Pas := F_mu__salt(T__salt_avg_conv_K)/ 1000;
  v__salt_conv := (m__salt / (rho__salt * Pi * (d__tube_ID_heat/2)
  ^2))/n__conv_parallel; #calculates if streams are in parallel
  Re__salt_conv := (rho__salt * v__salt_conv * d__tube_ID_heat)
  /mu__salt_conv_Pas;
  Pr__salt_conv := Cp__salt * mu__salt_conv_Pas / k__salt;
```

$$T_{salt\_avg\_conv} := 584.5259750$$

$$T_{salt\_avg\_conv\_K} := 857.6759750$$

$$\mu_{salt\_conv\_Pas} := 0.01170026601$$

$$v_{salt\_conv} := 0.04674769924$$

$$\Re_{salt\_conv} := 311.2446989$$

$$Pr_{salt\_conv} := 48.92838513 \quad \textbf{(4.2.11)}$$

```
> if Re__salt_conv > 10000 then
      Nu__salt_conv := 0.023 * Re__salt_conv^0.8 *
  Pr__salt_conv^0.4;
  elif Re__salt_conv > 2100 then
      Nu__salt_conv := 0.023 * Re__salt_conv^0.85 *
  Pr__salt_conv^0.4;
  else
      Nu__salt_conv := 1.86 * (Re__salt_conv * Pr__salt_conv *
  d__tube_ID_heat / L__tube_conv)^(1/3);
  end if;

  h__i_conv := Nu__salt_conv * k__salt / d__tube_ID_heat;
```

$$N_{salt\_conv} := 16.21227328$$

$$h_{i\_conv} := 234.6513238 \quad \textbf{(4.2.12)}$$

```
> #Checking U

  R__wall_conv := (d__tube_OD_conv/2) * ln
```

```
(d__tube_OD_conv/d__tube_ID_conv)/k__tube_heat;
R__i_conv := (d__tube_OD_conv/d__tube_ID_conv)*(1/h__i_conv);

#Back calculating (Claude helped with if/else statement)

R__inside_plus_wall := R__wall_conv + R__i_conv;
U__max_possible := 1 / R__inside_plus_wall;

if U__convection < U__max_possible then
    h__o_conv := 1 /(1/U__convection - R__wall_conv - R__i_conv);
    R__o_conv := 1 /h__o_conv;
    U__check_conv := U__convection;
    printf("  Flue gas side controls: h__o = %.1f W/m2/K\n",
evalf(h__o_conv));
else
    R__o_conv := 1 /U__convection;
    h__o_conv := U__convection;
    U__check_conv := 1 /(R__o_conv + R__wall_conv + R__i_conv);
    printf(" Salt-side resistance dominates\n");
    printf("  Walas U__c = %.1f W/m2/K but built-up U = %.1f
W/m2/K\n",evalf(U__convection), evalf(U__check_conv));
    printf("  --> Use U__check = %.1f for sizing\n", evalf
(U__check_conv));
end if;
```

$$R_{wall\_conv} := 0.0001816578367$$

$$R_{i\_conv} := 0.005524351088$$

$$R_{inside\_plus\_wall} := 0.005706008925$$

$$U_{max\_possible} := 175.2538443$$

$$h_{o\_conv} := 153.5528013$$

$$R_{o\_conv} := 0.006512417823$$

$$U_{check\_conv} := 81.84359744$$

```
Flue gas side controls: h__o = 153.6 W/m2/K
```

```
> U__check:= 1/(R__o_conv + R__wall_conv + R__i_conv);
  U__convection := U__convection;
```

$$U_{check} := 81.84359742$$

$$U_{convection} := 81.84359744 \quad \textbf{(4.2.13)}$$

```
> #=== Pressure drop calculation along length of pipes ===
```

```
> #===Convection Section===

  if Re__salt_conv > 4000 then
      f__conv := 0.316 / Re__salt_conv^(1/4);
  elif Re__salt_conv > 2100 then
      f__conv := 0.316 / Re__salt_conv^(1/4);  # approximate for
  transition
  else
      f__conv := 64 / Re__salt_conv;            # laminar
  end if;

  n__passes_total_conv := n__tubes_per_row_conv * n__rows_conv;
  L__total_straight_conv := n__passes_total_conv * L__tube_conv;
```

$$f_{conv} := 0.2056259921$$

$$n_{passes\_total\_conv} := 22$$

$$L_{total\_straight\_conv} := 19.22543648 \quad \textbf{(4.6)}$$

```
> # Bend counts
  n__bends_conv := n__passes_total_conv; #Assumed, a bend into the
  convection, and a bend into the radiat
```

$$n_{bends\_conv} := 22 \quad \textbf{(4.7)}$$

```
> # Loss coefficients (ESTIMATED!)
  K__bend := 1.5;
```

$$K_{bend} := 1.5 \quad \textbf{(4.8)}$$

```
> q__dyn_conv := 0.5 * rho__salt * v__salt_conv^2;
```

$$q_{dyn\_conv} := 2.239981068 \quad \textbf{(4.9)}$$

```
> DP__friction_conv := f__conv *
  (L__tube_conv_total/d__tube_ID_conv)*0.5*rho__salt*
  v__salt_conv^2; #Pa (Straight tubes)
  DP__bends_conv := n__bends_conv * K__bend * 0.5 * rho__salt *
  v__salt_conv^2;
  DP__total_conv := DP__friction_conv + DP__bends_conv; #Pa
```

$$DP_{friction\_conv} := 329.8470275$$

$$DP_{bends\_conv} := 73.91937525$$

$$DP_{total\_conv} := 403.7664028 \quad \textbf{(4.10)}$$

```
> #=== Radiation Section ===
```

```
> f__Darcy_rad := 0.316 / Re__salt^(1/4);
  f__coil := f__Darcy_rad * (1 + 3.6*(1 -
  d__tube_ID_heat/D__coil_heat)^0.8);
  DP__total_rad := f__coil *(L__tube_rad/d__tube_ID_heat)*0.5*
  rho__salt*v__salt^2;
```

$$f_{Darcy\_rad} := 0.04523456181$$

$$f_{coil} := 0.2031094100$$

$$DP_{total\_rad} := 22129.52550 \quad \textbf{(4.11)}$$

```
> DP__total := DP__total_rad + DP__total_conv;
```

$$DP_{total} := 22533.29190 \quad \textbf{(4.12)}$$

```
> #=== Energy Balance final Checks ===
```

```
> print("===Energy Check===");
  Q__total_check := Q__R_final + Q__convection;
  Q__absorbed:= Q__absorbed;
  print("===Energy release Check===");
  Q__released_check := Q__comb_gas + Q__comb_oil;
  Q__released_kW:= Q__released/1000;
  Q__required := Q__target;
  T__stack := T__stack;
```

"===Energy Check==="

$$Q_{total\_check} := 71008.21209$$

$$Q_{absorbed} := 68856.000$$

"===Energy release Check==="

$$Q_{released\_check} := 144.0852086$$

$$Q_{released\_kW} := 93.04864865$$

$$Q_{required} := 68.856$$

(4.13)

$$T_{stack} := 1258.980050 \quad \textbf{(4.13)}$$

```
> Q__required_release := Q__target * 1000 / eta__heater;   # W
    Q__from_oil := m__oil * LHV__oil * 1000;                  # W
    Q__from_factory_gas := (m__gas_factory / m__molar_fuel) *
  Q__mol_fuel * 1000;
    Q__shortfall := Q__required_release - Q__from_oil -
  Q__from_factory_gas;

    if Q__shortfall > 0 then
        m__gas_import := Q__shortfall / ((Q__mol_fuel /
  m__molar_fuel) * 1000);
    else
        m__gas_import := 0;
        printf("Oil + factory gas exceeds requirement. Consider
  reducing oil flow.\n");
    end if;
```

$$Q_{required\_release} := 93048.64865$$

$$Q_{from\_oil} := 102350.1050$$

$$Q_{from\_factory\_gas} := 41735.10364$$

$$Q_{shortfall} := -51036.55999$$

$$m_{gas\_import} := 0$$

```
Oil + factory gas exceeds requirement. Consider reducing oil flow.
```

```
> Prigg__ratio := evalf(A__p_gas / A__t);

  theta__oil_deg := evalf(theta__oil * 180 / Pi);
  spray_angle_full := evalf(2 * theta__oil_deg);
  SMD__oil_um := evalf(SMD_oil * 1e6);

  Q__comb_oil := evalf(m__oil * LHV__oil);
  SMD__corrected := evalf(SMD__oil_um * 1.75);
```

$$Prigg_{ratio} := 1.618681919$$

$$\theta_{oil\_deg} := 7.307468275$$

$$spray\_angle\_full := 14.61493655$$

$$SMD_{oil\_um} := 12.47760879$$

$$Q_{comb\_oil} := 102.3501050$$

$$SMD_{corrected} := 21.83581538 \quad \textbf{(4.14)}$$

==========
==========
==========
==========

==== Calculation Results ====
(Note, Claude was used to generate the results summary script)

```
> # ============================================================
  # SECTION 0: KEY SYSTEM OVERVIEW
  # ============================================================

  print("============================================================
  ======");
  print("          COMBINED BURNER HEATING SYSTEM — KEY OVERVIEW
        ");
  print("============================================================
  ======");

  printf("\n--- System Targets ---\n");
  printf("  Heat duty to salt:          %.1f kW\n", evalf
  (Q__target));
  printf("  Salt temperature range:     %.0f C  -->  %.0f C\n",
  evalf(T__salt_in), evalf(T__salt_out));
  printf("  Salt mass flow rate:        %.4f kg/s\n", evalf
  (m__salt));
  printf("  Required heat release:      %.1f kW  (at eta = %.0f%%)
  \n", evalf(Q__released/1000), evalf(eta__heater*100));

  printf("\n--- Energy Supply ---\n");
  printf("  Gas burner (HHV):           %.2f kW\n", evalf
  (Q__comb_gas));
  printf("  Oil burner (LHV):           %.2f kW\n", evalf
  (Q__comb_oil));
  printf("  Total fuel available:       %.2f kW\n", evalf
  (Q__comb_gas + Q__comb_oil));
  printf("  Energy shortfall:           %.2f kW\n", evalf
  (Q__released/1000 - Q__comb_gas - Q__comb_oil));
```

```
if evalf(Q__comb_gas + Q__comb_oil) < evalf(Q__released/1000)
then
    printf("  ENERGY BALANCE:              *** FAIL ***\n");
    printf("  --> Solve for m__gas_import to close the %.1f kW
gap\n", evalf(Q__released/1000 - Q__comb_gas - Q__comb_oil));
else
    printf("  ENERGY BALANCE:              PASS\n");
end if;

printf("\n--- Heat Distribution ---\n");
printf("  Radiant fraction (assumed): %.3f\n", evalf(f__radiant))
;
printf("  Radiant fraction (calc):    %.3f\n", evalf
(Q__R_final/Q__absorbed));
printf("  Radiant zone absorption:    %.2f kW\n", evalf
(Q__R_final/1000));
printf("  Convection zone absorption: %.2f kW\n", evalf
(Q__convection/1000));
printf("  Total absorbed (check):     %.2f kW  (target: %.1f kW)
\n", evalf((Q__R_final + Q__convection)/1000), evalf(Q__target));
printf("  Thermal efficiency:         %.1f %%\n", evalf
(eta__heater*100));
printf("  Excess air:                 %.0f %%\n", evalf
(z__excess*100));

if evalf(abs(Q__R_final/Q__absorbed - f__radiant)) > 0.05 then
    printf("  f_radiant CONVERGENCE:      *** NOT CONVERGED —
update to %.3f ***\n", evalf(Q__R_final/Q__absorbed));
else
    printf("  f_radiant CONVERGENCE:      PASS\n");
end if;

printf("\n--- Critical Status Flags ---\n");

if evalf(Q__comb_gas + Q__comb_oil) >= evalf(Q__released/1000)
then
    printf("  [1] Energy balance:         PASS\n");
else
    printf("  [1] Energy balance:         FAIL\n");
end if;

if evalf(Prigg__ratio) >= 1.5 and evalf(Prigg__ratio) <= 2.2 then
    printf("  [2] Gas Prigg ratio:        PASS  (%.1f)\n", evalf
(Prigg__ratio));
else
```

```
    printf("  [2] Gas Prigg ratio:        FAIL  (%.1f, need 1.5
-2.2)\n", evalf(Prigg__ratio));
end if;

if evalf(K__oil) >= 0.19 and evalf(K__oil) <= 1.21 then
    printf("  [3] Oil K parameter:        PASS  (%.3f)\n", evalf
(K__oil));
else
    printf("  [3] Oil K parameter:        FAIL  (%.3f, need 0.19
-1.21)\n", evalf(K__oil));
end if;

if evalf(abs(q__rad_check - q__rad)/q__rad) <= 0.10 then
    printf("  [4] Radiant flux:           PASS  (%.0f W/m2)\n",
evalf(q__rad_check));
else
    printf("  [4] Radiant flux:           FAIL  (%.0f vs %.0f
W/m2)\n", evalf(q__rad_check), evalf(q__rad));
end if;

if evalf(T__skin_max) <= evalf(T__salt_max) then
    printf("  [5] Skin temperature:       PASS  (%.1f C)\n",
evalf(T__skin_max));
else
    printf("  [5] Skin temperature:       FAIL  (%.1f vs %.1f C)
\n", evalf(T__skin_max), evalf(T__salt_max));
end if;

if evalf(A__rad_actual) >= evalf(A__required_UA) then
    printf("  [6] Radiant area:           PASS\n");
else
    printf("  [6] Radiant area:           FAIL  (%.3f vs %.3f m2)
\n", evalf(A__rad_actual), evalf(A__required_UA));
end if;

if evalf(abs(Q__R_final/Q__absorbed - f__radiant)) <= 0.05 then
    printf("  [7] f_radiant converged:    PASS\n");
else
    printf("  [7] f_radiant converged:    FAIL\n");
end if;

if evalf(SMD__corrected) <= 100 then
    printf("  [8] Oil SMD:                PASS  (%.1f um)\n",
evalf(SMD__corrected));
else
```

```
    printf("  [8] Oil SMD:                FAIL  (%.1f um, need <
100)\n", evalf(SMD__corrected));
end if;

# ============================================================
# SECTION 1: GAS BURNER DESIGN SUMMARY
# ============================================================

print("============================================================
======");
print("==== SECTION 1: GAS BURNER DESIGN ====");



printf("\n--- Fuel Supply ---\n");
printf("  Factory biogas:             %.6f kg/s\n", evalf
(m__gas_factory));
printf("  Imported biogas:            %.6f kg/s\n", evalf
(m__gas_import));
printf("  Total gas flow:             %.6f kg/s\n", evalf
(m__fuel_gas));
printf("  Biogas composition:         %.0f%% CH4 / %.0f%% CO2
(molar)\n", evalf(xi__CH4*100), evalf(xi__CO2*100));
printf("  Biogas density:             %.4f kg/m3\n", evalf
(rho__mix));
printf("  Volumetric flow:            %.6f m3/h\n", evalf
(V__fuel));
printf("  Molar mass of fuel mix:     %.5f kg/mol\n", evalf
(m__molar_fuel));
printf("  Combustion heat (HHV):      %.2f kW\n", evalf
(Q__comb_gas));

printf("\n--- Injector Orifice ---\n");
printf("  Orifice area:               %.4e m2\n", evalf(A__o));
printf("  Orifice radius:             %.4f mm\n", evalf(r__o*
1000));
printf("  Orifice diameter:           %.4f mm\n", evalf(d__o*
1000));
printf("  Bore diameter (1.75r):      %.4f mm\n", evalf(b__o*
1000));
printf("  Orifice velocity:           %.2f m/s\n", evalf(v__o));
printf("  Discharge coefficient:      %.2f\n", evalf(C__d_gas));

printf("\n--- Air Entrainment ---\n");
printf("  Stoich. vol. AFR:           %.3f m3air/m3gas\n", evalf
```

```
(v__air));
printf("  Air volume flow:            %.6f m3/h\n", evalf(V__air)
);
printf("  Entrainment ratio:          %.3f  (target ~%.3f)\n",
evalf(r_ent), evalf(v__air*0.5));
printf("  Air mass flow:              %.6f kg/s\n", evalf
(m__air_g_s));
printf("  Specific gravity (s):       %.4f\n", evalf(s));

printf("\n--- Mixing Throat ---\n");
printf("  Throat diameter:            %.4f mm\n", evalf(d__t*
1000));
printf("  Throat length:              %.4f mm\n", evalf(L__t*
1000));
printf("  Throat area:                %.4e m2\n", evalf(A__t));
printf("  Reynolds number:            %.0f\n", evalf(Re__t));
printf("  Friction factor:            %.6f\n", evalf(f__t));
printf("  Pressure drop:              %.2f Pa\n", evalf
(DP__t_gas));
printf("  Mixture temperature:        %.1f K\n", evalf(T__t));
printf("  Mixture density:            %.4f kg/m3\n", evalf
(rho__t));
printf("  Mixture viscosity:          %.4e Pa.s\n", evalf(mu__t))
;

printf("\n--- Flame Ports ---\n");
printf("  Port diameter:              %.1f mm\n", evalf(d__p_gas*
1000));
printf("  Number of ports:            %d\n", n__p_gas);
printf("  Total port area:            %.4e m2\n", evalf(A__p_gas)
);
printf("  Port velocity:              %.4f m/s\n", evalf
(v__p_gas));

printf("\n--- Gas Burner Checks ---\n");

printf("  Prigg ratio (Ap/At):        %.2f  (valid: 1.5-2.2)  ",
evalf(Prigg__ratio));
if evalf(Prigg__ratio) >= 1.5 and evalf(Prigg__ratio) <= 2.2 then
    printf("PASS\n");
else
    printf("FAIL\n");
end if;

printf("  Throat dP vs supply:        %.2f Pa vs %.0f Pa  ",
```

```
evalf(DP__t_gas), evalf(p__prior_gas*100));
if evalf(DP__t_gas) < evalf(p__prior_gas*100) then
    printf("PASS\n");
else
    printf("FAIL\n");
end if;

# ============================================================
# SECTION 2: OIL BURNER DESIGN SUMMARY
# ============================================================

print("============================================================
======");
print("==== SECTION 2: OIL BURNER DESIGN ====");



printf("\n--- Bio-Oil Properties ---\n");
printf("  Mass flow rate:             %.6f kg/s  (%.2f kg/h)\n",
evalf(m__oil), evalf(m__oil*3600));
printf("  Empirical formula:          CH_%.2f O_%.3f N_%.4f\n",
evalf(n__H), evalf(n__O), evalf(n__N));
printf("  Density:                    %.0f kg/m3\n", evalf
(rho__oil));
printf("  Dynamic viscosity:          %.4f Pa.s  (%.1f cSt)\n",
evalf(mu__oil), evalf(nu__oil*1e6));
printf("  Surface tension:            %.1f mN/m\n", evalf
(sigma__oil*1000));
printf("  HHV / LHV:                  %.0f / %.0f kJ/kg\n", evalf
(HHV__oil), evalf(LHV__oil));

printf("\n--- Atomizer Geometry ---\n");
printf("  Nozzle diameter (D0):       %.2f mm\n", evalf(D__0*
1000));
printf("  Swirl chamber dia (Ds):     %.2f mm\n", evalf(D__s*
1000));
printf("  Passage diameter (Dp):      %.2f mm\n", evalf(D__p*
1000));
printf("  Swirl chamber length (Ls):  %.2f mm\n", evalf(L__s*
1000));
printf("  Nozzle length (L0):         %.2f mm\n", evalf(L__0*
1000));
printf("  Passage length (Lp):        %.2f mm\n", evalf(L__p*
1000));
printf("  Number of passages:         %d\n", n__p_oil);
```

```
printf("  Total passage area (Ap):    %.4e m2\n", evalf(A__p_oil)
);
printf("  Nozzle area (A0):           %.4e m2\n", evalf(A__0));
printf("  Injection pressure:         %.0f bar\n", evalf
(DP__oil/1e5));

printf("\n--- Flow Characteristics ---\n");
printf("  Flow number (FN):           %.3e m2\n", evalf(FN__oil))
;
printf("  Discharge coeff (Cd):       %.4f\n", evalf(Cd__oil));
printf("  Cd Carlisle:                %.4f\n", rhs
(eq1__Carlisle_oil));
printf("  Cd Rizk-Lefebvre:           %.4f\n", rhs(eq2__RL_oil));
printf("  Cd Jones:                   %.4f\n", rhs
(eq3__Jones_oil));
printf("  Liquid exit velocity (U0):  %.2f m/s\n", evalf
(U__0_oil));

printf("\n--- Spray Performance ---\n");
printf("  Air core fraction (X):      %.4f\n", evalf(X__oil));
printf("  Spray semi-angle:           %.2f deg\n", evalf
(theta__oil_deg));
printf("  Full cone angle:            %.2f deg\n", evalf
(spray_angle_full));
printf("  Sheet thickness (h0):       %.2f um\n", evalf(h__0_oil*
1e6));
printf("  Ligament diameter (DL):     %.2f um\n", evalf(D__L_oil*
1e6));
printf("  SMD (theoretical):          %.1f um\n", evalf
(SMD__oil_um));
printf("  SMD (corrected x1.75):      %.1f um\n", evalf
(SMD__corrected));

printf("\n--- Heat Output ---\n");
printf("  Combustion heat (LHV):      %.2f kW\n", evalf
(Q__comb_oil));

printf("\n--- Oil Burner Checks ---\n");

printf("  K = Ap/(Ds*D0):             %.4f  (valid: 0.19-1.21)
", evalf(K__oil));
if evalf(K__oil) >= 0.19 and evalf(K__oil) <= 1.21 then
    printf("PASS\n");
else
    printf("FAIL\n");
```

```
end if;

printf("  Ds/D0:                          %.1f   (valid: 1.41-8.13)
", evalf(D__s/D__0));
if evalf(D__s/D__0) >= 1.41 and evalf(D__s/D__0) <= 8.13 then
    printf("PASS\n");
else
    printf("FAIL\n");
end if;

printf("  Ls/Ds:                          %.2f   (must be > 0.5)   ",
evalf(L__s/D__s));
if evalf(L__s/D__s) > 0.5 then
    printf("PASS\n");
else
    printf("FAIL\n");
end if;

printf("  L0/D0:                          %.2f\n", evalf(L__0/D__0));

printf("  Lp/Dp:                          %.2f   (must be > 1.3)   ",
evalf(L__p/D__p));
if evalf(L__p/D__p) > 1.3 then
    printf("PASS\n");
else
    printf("FAIL\n");
end if;

printf("  SMD corrected:                  %.1f um   (need < 100)   ",
evalf(SMD__corrected));
if evalf(SMD__corrected) <= 100 then
    printf("PASS\n");
else
    printf("FAIL\n");
end if;

# ============================================================
# SECTION 3: RADIANT ZONE — HELICAL COIL
# ============================================================

print("============================================================
======");
print("==== SECTION 3: RADIANT ZONE ====");

printf("\n--- Heater Shell ---\n");
```

```
printf("  Shell outer diameter:       %.0f mm\n", evalf
(D__shell_heat*1000));
printf("  Refractory thickness:       %.0f mm\n", evalf
(t__refr_heat*1000));
printf("  Refractory conductivity:    %.2f W/m/K\n", evalf
(k__refr_heat));
printf("  Firebox core diameter:      %.0f mm\n", evalf
(D__firebox*1000));
printf("  Firebox L/D ratio:          %.2f\n", evalf(L_D__ratio))
;

printf("\n--- Helical Coil Geometry ---\n");
printf("  Tube OD / ID:               %.1f / %.1f mm\n", evalf
(d__tube_OD_heat*1000), evalf(d__tube_ID_heat*1000));
printf("  Tube conductivity:          %.0f W/m/K\n", evalf
(k__tube_heat));
printf("  Coil helix diameter:        %.0f mm\n", evalf
(D__coil_heat*1000));
printf("  Coil pitch:                 %.0f mm\n", evalf
(p__coil_heat*1000));
printf("  Pitch/OD ratio:             %.2f\n", evalf(x__spacing))
;
printf("  Turns (exact):              %.2f\n", evalf
(N__turns_heat));
printf("  Turns (integer):            %d\n", N__turns_int_heat);
printf("  Coil height:                %.3f m\n", evalf(H__coil));
printf("  Turn length:                %.4f m\n", evalf
(L__turn_heat));
printf("  Total tube length:          %.2f m\n", evalf
(L__tube_rad));

printf("\n--- Surface Areas ---\n");
printf("  Required radiant area:      %.4f m2\n", evalf
(A__radiant));
printf("  Actual radiant area:        %.4f m2\n", evalf
(A__rad_actual));
printf("  Cold plane area (Acp):      %.4f m2\n", evalf(A__cp));
printf("  Shell area:                 %.4f m2\n", evalf(A__shell)
);
printf("  Refractory area (Aw):       %.4f m2\n", evalf(A__refr))
;
printf("  Tube absorptivity:          %.4f\n", evalf(alpha__tube)
);
printf("  Effective area (aAR):       %.4f m2\n", evalf
(alpha__AR));
```

```
printf("  Aw / alpha*AR:              %.4f\n", evalf
(ratio__Aw_alphaAR));

printf("\n--- Flue Gas ---\n");
printf("  Total air flow:             %.5f kg/s\n", evalf
(m__total_air));
printf("  Total flue gas flow:        %.5f kg/s\n", evalf
(m__flue));
printf("  Converged gas temp (Tg):    %.1f C\n", evalf(T__g_C));
printf("  Mean beam length:           %.4f m\n", evalf(L__beam));
printf("  P(CO2+H2O):                 %.4f atm\n", evalf
(P__CO2_H2O));
printf("  PL product:                 %.4f atm.ft\n", evalf(PL));
printf("  Gas emissivity:             %.4f\n", evalf
(phi__g_final));
printf("  Exchange factor (F):        %.4f\n", evalf
(F__exc_final));
printf("  Iterations:                 %d\n", iter);

printf("\n--- Radiant Flux ---\n");
printf("  Target flux:                %.0f W/m2\n", evalf(q__rad)
);
printf("  Actual flux:                %.0f W/m2\n", evalf
(q__rad_check));
printf("  Deviation:                  %.1f %%  ", evalf(abs
(q__rad_check - q__rad)/q__rad*100));
if evalf(abs(q__rad_check - q__rad)/q__rad) <= 0.10 then
    printf("PASS\n");
else
    printf("FAIL\n");
end if;

printf("\n--- Salt-Side (Radiant) ---\n");
printf("  Salt avg temperature:       %.1f C\n", evalf
(T__salt_avg_K - 273.15));
printf("  Salt viscosity:             %.5f Pa.s\n", evalf
(mu__salt_Pas));
printf("  Salt velocity:              %.3f m/s\n", evalf(v__salt)
);
printf("  Reynolds number:            %.0f\n", evalf(Re__salt));
printf("  Prandtl number:             %.1f\n", evalf(Pr__salt));
printf("  Dean number:                %.0f\n", evalf(De__salt));
printf("  Nusselt number:             %.1f\n", evalf(Nu__salt));
printf("  h_i (inside film):          %.1f W/m2/K\n", evalf(h__i)
);
```

```
printf("\n--- Overall HT (Radiant) ---\n");
printf("  Mean tube wall temp:        %.1f C\n", evalf(T__tube_C)
);
printf("  h_o (outside film):         %.1f W/m2/K\n", evalf(h__o)
);
printf("  R_outside:                  %.6f m2.K/W\n", evalf(R__o)
);
printf("  R_wall:                     %.6f m2.K/W\n", evalf
(R__wall));
printf("  R_inside:                   %.6f m2.K/W\n", evalf(R__i)
);
printf("  U_overall:                  %.1f W/m2/K\n", evalf
(U__overall));
printf("  LMTD:                       %.1f K\n", evalf(LMTD__rad)
);

printf("\n--- Area Verification ---\n");
printf("  Area required (UA):         %.4f m2\n", evalf
(A__required_UA));
printf("  Area provided:              %.4f m2  ", evalf
(A__rad_actual));
if evalf(A__required_UA) <= evalf(A__rad_actual) then
    printf("PASS (%.1f%% margin)\n", evalf(
(A__rad_actual/A__required_UA - 1)*100));
else
    printf("FAIL (%.1f%% short)\n", evalf((1 -
A__rad_actual/A__required_UA)*100));
end if;

printf("\n--- Skin Temperature ---\n");
printf("  Peak inside skin:           %.1f C\n", evalf
(T__skin_max));
printf("  Peak outside skin:          %.1f C\n", evalf
(T__skin_outside));
printf("  Salt max allowed:           %.1f C  ", evalf
(T__salt_max));
if evalf(T__skin_max) <= evalf(T__salt_max) then
    printf("PASS (%.1f C margin)\n", evalf(T__salt_max -
T__skin_max));
else
    printf("FAIL (%.1f C over)\n", evalf(T__skin_max -
T__salt_max));
end if;

printf("\n--- Convergence ---\n");
```

```
printf("  Q_R target:                  %.2f kW\n", evalf
(Q__radiant/1000));
printf("  Q_R calculated:              %.2f kW\n", evalf
(Q__R_final/1000));
printf("  Deviation:                   %.1f %%\n", evalf(abs
(Q__R_final - Q__radiant)/Q__radiant*100));
if evalf(abs(Q__R_final/Q__absorbed - f__radiant)) > 0.05 then
    printf("  f_radiant:                     NOT CONVERGED (update
to %.3f)\n", evalf(Q__R_final/Q__absorbed));
else
    printf("  f_radiant:                     CONVERGED\n");
end if;

# ============================================================
# SECTION 4: CONVECTION ZONE
# ============================================================

print("============================================================
======");
print("==== SECTION 4: CONVECTION ZONE ====");

printf("\n--- Convection Duty ---\n");
printf("  Heat load:                   %.2f kW\n", evalf
(Q__convection/1000));
printf("  Salt inlet temp:             %.0f C\n", evalf
(T__salt_in));
printf("  Salt rad zone inlet:         %.1f C\n", evalf
(T__salt_rad_in_C));
printf("  Stack temperature:           %.1f F  (%.1f C)\n", evalf
(T__stack), evalf((T__stack-32)*5/9));

printf("\n--- Duct Geometry ---\n");
printf("  Duct width (square):         %.4f m\n", evalf(W__conv));
printf("  Tube OD / ID:                %.1f / %.1f mm\n", evalf
(d__tube_OD_conv*1000), evalf(d__tube_ID_conv*1000));
printf("  Horizontal pitch:            %.1f mm\n", evalf
(p__conv_horizontal*1000));
printf("  Vertical pitch:              %.1f mm\n", evalf
(p__conv_vertical*1000));
printf("  Tubes per row:               %d\n",
n__tubes_per_row_conv);
printf("  Number of rows:              %d\n", n__rows_conv);
printf("  Shield rows:                 %d\n", n__shield_rows);
printf("  Parallel salt streams:       %d\n", n__conv_parallel);
printf("  Tube length per pass:        %.4f m\n", evalf
```

```
(L__tube_conv));
printf("  Total tube length:          %.2f m\n", evalf
(L__tube_conv_total));
printf("  Bend allowance:             %.1f mm\n", evalf
(W__conv_bend*1000));

printf("\n--- Flow Parameters ---\n");
printf("  Free flow area:             %.5f m2\n", evalf
(A__conv_free));
printf("  Mass velocity (G):          %.4f lb/s/ft2\n", evalf
(G__convection));
printf("  LMTD (convection):          %.1f F\n", evalf
(LMTD__conv_F));
printf("  DT_hot:                     %.1f F\n", evalf
(DT__conv_hot));
printf("  DT_cold:                    %.1f F\n", evalf
(DT__conv_cold));

printf("\n--- Heat Transfer Coefficients ---\n");
printf("  Walas Uc (bare tube):       %.2f Btu/hr/ft2/F\n", evalf
(U__convection_Btu));
printf("  Walas Uc (SI):              %.1f W/m2/K\n", evalf
(U__convection));
printf("  U_check (built-up):         %.1f W/m2/K\n", evalf
(U__check));

printf("\n--- Area Sizing ---\n");
printf("  Required area:              %.4f m2\n", evalf
(A__convection));
printf("  Area per row:               %.4f m2\n", evalf
(A__conv_per_row));

printf("\n--- Salt-Side (Convection) ---\n");
printf("  Salt avg temp:              %.1f C\n", evalf
(T__salt_avg_conv));
printf("  Salt viscosity:             %.5f Pa.s\n", evalf
(mu__salt_conv_Pas));
printf("  Salt velocity:              %.3f m/s\n", evalf
(v__salt_conv));
printf("  Reynolds number:            %.0f\n", evalf
(Re__salt_conv));
printf("  Prandtl number:             %.1f\n", evalf
(Pr__salt_conv));
printf("  Nusselt number:             %.1f\n", evalf
(Nu__salt_conv));
```

```
printf("  h_i (inside film):          %.1f W/m2/K\n", evalf
(h__i_conv));
printf("  h_o (outside film):         %.1f W/m2/K\n", evalf
(h__o_conv));

printf("\n--- Resistance Breakdown ---\n");
printf("  R_outside (flue gas):       %.6f m2.K/W\n", evalf
(R__o_conv));
printf("  R_wall:                     %.6f m2.K/W\n", evalf
(R__wall_conv));
printf("  R_inside (salt):            %.6f m2.K/W\n", evalf
(R__i_conv));

# ============================================================
# SECTION 5: SALT-SIDE PRESSURE DROP
# ============================================================

print("============================================================
======");
print("==== SECTION 5: PRESSURE DROP ====");

printf("\n--- Radiation Section ---\n");
printf("  Coil friction factor:       %.4f\n", evalf(f__coil));
printf("  Tube length (rad):          %.2f m\n", evalf
(L__tube_rad));
printf("  Pressure drop (rad):        %.0f Pa  (%.2f bar)\n",
evalf(DP__total_rad), evalf(DP__total_rad/1e5));

printf("\n--- Convection Section ---\n");
printf("  Friction factor:            %.5f\n", evalf(f__conv));
printf("  Straight tube length:       %.2f m\n", evalf
(L__total_straight_conv));
printf("  Friction dP:                %.0f Pa\n", evalf
(DP__friction_conv));
printf("  Bend loss coeff (K):        %.1f\n", evalf(K__bend));
printf("  NOTE: n__bends_conv not assigned. Add:\n");
printf("        n__bends_conv := n__bends_ret_conv;\n");

printf("\n--- Total ---\n");
printf("  Rad section dP:             %.0f Pa\n", evalf
(DP__total_rad));
printf("  Conv section dP:            %.0f Pa\n", evalf
(DP__total_conv));
```

```
  print("============================================================
  ======");
  print("                END OF DESIGN SUMMARY
       ");
  print("============================================================
  ======");

  "================================================================"
     "      COMBINED BURNER HEATING SYSTEM — KEY OVERVIEW       "
  "================================================================"

--- System Targets ---
  Heat duty to salt:          68.9 kW
  Salt temperature range:     581 C  -->  620 C
  Salt mass flow rate:        0.7608 kg/s
  Required heat release:      93.0 kW  (at eta = 74%)

--- Energy Supply ---
  Gas burner (HHV):           41.74 kW
  Oil burner (LHV):           102.35 kW
  Total fuel available:       144.09 kW
  Energy shortfall:           -51.04 kW
  ENERGY BALANCE:             PASS

--- Heat Distribution ---
  Radiant fraction (assumed): 0.803
  Radiant fraction (calc):    0.834
  Radiant zone absorption:    57.44 kW
  Convection zone absorption: 13.56 kW
  Total absorbed (check):     71.01 kW  (target: 68.9 kW)
  Thermal efficiency:         74.0 %
  Excess air:                 15 %
  f_radiant CONVERGENCE:      PASS

--- Critical Status Flags ---
  [1] Energy balance:         PASS
  [2] Gas Prigg ratio:        PASS  (1.6)
  [3] Oil K parameter:        PASS  (0.251)
  [4] Radiant flux:           PASS  (10099 W/m2)
  [5] Skin temperature:       PASS  (632.4 C)
  [6] Radiant area:           PASS
  [7] f_radiant converged:    PASS
  [8] Oil SMD:                PASS  (21.8 um)

  "================================================================"
```

"==== SECTION 1: GAS BURNER DESIGN ===="

```
--- Fuel Supply ---
  Factory biogas:             0.002126 kg/s
  Imported biogas:            0.000000 kg/s
  Total gas flow:             0.002126 kg/s
  Biogas composition:         60% CH4 / 40% CO2 (molar)
  Biogas density:             0.9167 kg/m3
  Volumetric flow:            8.347302 m3/h
  Molar mass of fuel mix:     0.02723 kg/mol
  Combustion heat (HHV):      41.74 kW

--- Injector Orifice ---
  Orifice area:               9.8230e-06 m2
  Orifice radius:             1.7687 mm
  Orifice diameter:           3.5374 mm
  Bore diameter (1.75r):      3.0952 mm
  Orifice velocity:           236.05 m/s
  Discharge coefficient:      0.90

--- Air Entrainment ---
  Stoich. vol. AFR:           5.714 m3air/m3gas
  Air volume flow:            47.698866 m3/h
  Entrainment ratio:          2.857  (target ~2.857)
  Air mass flow:              0.015688 kg/s
  Specific gravity (s):       0.7484

--- Mixing Throat ---
  Throat diameter:            15.2207 mm
  Throat length:              152.2071 mm
  Throat area:                1.8186e-04 m2
  Reynolds number:            37137
  Friction factor:            0.022763
  Pressure drop:              232.60 Pa
  Mixture temperature:        307.1 K
  Mixture density:            0.8450 kg/m3
  Mixture viscosity:          1.7032e-05 Pa.s

--- Flame Ports ---
  Port diameter:              5.0 mm
  Number of ports:            15
  Total port area:            2.9438e-04 m2
  Port velocity:              30.3814 m/s

--- Gas Burner Checks ---
```

```
  Prigg ratio (Ap/At):        1.62  (valid: 1.5-2.2)  PASS
  Throat dP vs supply:        232.60 Pa vs 3000 Pa  PASS

  "================================================================="
```

"==== SECTION 2: OIL BURNER DESIGN ===="

```
--- Bio-Oil Properties ---
  Mass flow rate:             0.003486 kg/s  (12.55 kg/h)
  Empirical formula:          CH_1.43 O_0.332 N_0.0013
  Density:                    1200 kg/m3
  Dynamic viscosity:          0.0720 Pa.s  (60.0 cSt)
  Surface tension:            30.0 mN/m
  HHV / LHV:                  31030 / 29360 kJ/kg

--- Atomizer Geometry ---
  Nozzle diameter (D0):       1.50 mm
  Swirl chamber dia (Ds):     3.00 mm
  Passage diameter (Dp):      0.60 mm
  Swirl chamber length (Ls):  2.00 mm
  Nozzle length (L0):         1.00 mm
  Passage length (Lp):        1.00 mm
  Number of passages:         4
  Total passage area (Ap):    1.1304e-06 m2
  Nozzle area (A0):           1.7662e-06 m2
  Injection pressure:         10 bar

--- Flow Characteristics ---
  Flow number (FN):           1.006e-07 m2
  Discharge coeff (Cd):       0.0403
  Cd Carlisle:                0.0309
  Cd Rizk-Lefebvre:           0.2481
  Cd Jones:                   0.2222
  Liquid exit velocity (U0):  40.82 m/s

--- Spray Performance ---
  Air core fraction (X):      0.9597
  Spray semi-angle:           7.31 deg
  Full cone angle:            14.61 deg
  Sheet thickness (h0):       18.86 um
  Ligament diameter (DL):     6.60 um
  SMD (theoretical):          12.5 um
  SMD (corrected x1.75):      21.8 um

--- Heat Output ---
  Combustion heat (LHV):      102.35 kW
```

```
--- Oil Burner Checks ---
  K = Ap/(Ds*D0):              0.2512  (valid: 0.19-1.21)  PASS
  Ds/D0:                       2.0  (valid: 1.41-8.13)  PASS
  Ls/Ds:                       0.67  (must be > 0.5)  PASS
  L0/D0:                       0.67
  Lp/Dp:                       1.67  (must be > 1.3)  PASS
  SMD corrected:               21.8 um  (need < 100)  PASS

  "============================================================"
```

"==== SECTION 3: RADIANT ZONE ===="

```
--- Heater Shell ---
  Shell outer diameter:        1250 mm
  Refractory thickness:        150 mm
  Refractory conductivity:     0.35 W/m/K
  Firebox core diameter:       952 mm
  Firebox L/D ratio:           1.36

--- Helical Coil Geometry ---
  Tube OD / ID:                48.0 / 38.0 mm
  Tube conductivity:           25 W/m/K
  Coil helix diameter:         1000 mm
  Coil pitch:                  108 mm
  Pitch/OD ratio:              2.24
  Turns (exact):               11.66
  Turns (integer):             12
  Coil height:                 1.290 m
  Turn length:                 3.1434 m
  Total tube length:           37.72 m

--- Surface Areas ---
  Required radiant area:       5.5291 m2
  Actual radiant area:         5.6882 m2
  Cold plane area (Acp):       4.0527 m2
  Shell area:                  7.5202 m2
  Refractory area (Aw):        3.4675 m2
  Tube absorptivity:           0.8232
  Effective area (aAR):        3.3362 m2
  Aw / alpha*AR:               1.0394

--- Flue Gas ---
  Total air flow:              0.04972 kg/s
  Total flue gas flow:         0.05534 kg/s
  Converged gas temp (Tg):     810.0 C
  Mean beam length:            0.9520 m
```

```
  P(CO2+H2O):                0.2557 atm
  PL product:                0.7986 atm.ft
  Gas emissivity:            0.2733
  Exchange factor (F):       0.4154
  Iterations:                4

--- Radiant Flux ---
  Target flux:               10000 W/m2
  Actual flux:               10099 W/m2
  Deviation:                 1.0 %  PASS

--- Salt-Side (Radiant) ---
  Salt avg temperature:      600.3 C
  Salt viscosity:            0.01070 Pa.s
  Salt velocity:             0.327 m/s
  Reynolds number:           2382
  Prandtl number:            44.8
  Dean number:               464
  Nusselt number:            56.3
  h_i (inside film):         814.7 W/m2/K

--- Overall HT (Radiant) ---
  Mean tube wall temp:       659.8 C
  h_o (outside film):        67.2 W/m2/K
  R_outside:                 0.014877 m2.K/W
  R_wall:                    0.000224 m2.K/W
  R_inside:                  0.001551 m2.K/W
  U_overall:                 60.1 W/m2/K
  LMTD:                      205.4 K

--- Area Verification ---
  Area required (UA):        4.4827 m2
  Area provided:             5.6882 m2  PASS (26.9% margin)

--- Skin Temperature ---
  Peak inside skin:          632.4 C
  Peak outside skin:         634.7 C
  Salt max allowed:          650.0 C  PASS (17.6 C margin)

--- Convergence ---
  Q_R target:                55.29 kW
  Q_R calculated:            57.44 kW
  Deviation:                 3.9 %
  f_radiant:                 CONVERGED
```

"=="

"==== SECTION 4: CONVECTION ZONE ===="

```
--- Convection Duty ---
  Heat load:                  13.56 kW
  Salt inlet temp:            581 C
  Salt rad zone inlet:        588.4 C
  Stack temperature:          1259.0 F  (681.7 C)

--- Duct Geometry ---
  Duct width (square):        0.8839 m
  Tube OD / ID:               35.0 / 27.0 mm
  Horizontal pitch:           87.5 mm
  Vertical pitch:             87.5 mm
  Tubes per row:              11
  Number of rows:             2
  Shield rows:                2
  Parallel salt streams:      7
  Tube length per pass:       0.8739 m
  Total tube length:          19.34 m
  Bend allowance:             5.0 mm

--- Flow Parameters ---
  Free flow area:             0.43597 m2
  Mass velocity (G):          0.0260 lb/s/ft2
  LMTD (convection):          276.3 F
  DT_hot:                     398.9 F
  DT_cold:                    181.8 F

--- Heat Transfer Coefficients ---
  Walas Uc (bare tube):       14.41 Btu/hr/ft2/F
  Walas Uc (SI):              81.8 W/m2/K
  U_check (built-up):         81.8 W/m2/K

--- Area Sizing ---
  Required area:              1.0798 m2
  Area per row:               1.0564 m2

--- Salt-Side (Convection) ---
  Salt avg temp:              584.5 C
  Salt viscosity:             0.01170 Pa.s
  Salt velocity:              0.047 m/s
  Reynolds number:            311
  Prandtl number:             48.9
  Nusselt number:             16.2
```

```
  h_i (inside film):          234.7 W/m2/K
  h_o (outside film):         153.6 W/m2/K

--- Resistance Breakdown ---
  R_outside (flue gas):       0.006512 m2.K/W
  R_wall:                     0.000182 m2.K/W
  R_inside (salt):            0.005524 m2.K/W

  "============================================================="
                "==== SECTION 5: PRESSURE DROP ===="

--- Radiation Section ---
  Coil friction factor:       0.2031
  Tube length (rad):          37.72 m
  Pressure drop (rad):        22130 Pa  (0.22 bar)

--- Convection Section ---
  Friction factor:            0.20563
  Straight tube length:       19.23 m
  Friction dP:                330 Pa
  Bend loss coeff (K):        1.5
  NOTE: n__bends_conv not assigned. Add:
        n__bends_conv := n__bends_ret_conv;

--- Total ---
  Rad section dP:             22130 Pa
  Conv section dP:            404 Pa

  "============================================================="
            "       END OF DESIGN SUMMARY                "
  "============================================================="
```

(5.1)